



FULL STACK DEVELOPMENT

QUALIDADE DE SOFTWARE

AULA 03

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	8
REFERÊNCIAS.....	9

EMANIP

O QUE VEM POR AÍ?

Nesta aula, aprofundaremos o conhecimento sobre o Desenvolvimento Orientado a Testes (TDD), uma metodologia que tem como foco a criação de testes antes da implementação do código.

Veremos ainda os princípios e benefícios do TDD, bem como entenderemos como aplicá-lo em nosso processo de desenvolvimento.



HANDS ON

Nesta aula aprenderemos, na prática, como aplicar o TDD, demonstrando como aplicá-lo a uma nova funcionalidade.

Começaremos escrevendo um teste que descreverá o comportamento esperado dessa funcionalidade: veremos as falhas e, posteriormente, criaremos o código para implementação e o refatoraremos em seguida.



SAIBA MAIS

Aplicando TDD na prática

Na aula anterior, discutimos a importância dos testes de unidade e como eles podem ser utilizados para validar novas funcionalidades em um código existente. Agora, vamos dar um passo adiante e explorar o Desenvolvimento Orientado a Testes (ou TDD, Test-Driven Development).

Ele se configura como uma abordagem de desenvolvimento de software que se concentra na criação de testes antes da implementação do código. Assim, nesta aula, aprenderemos os princípios básicos do TDD e como aplicá-los em nosso fluxo de trabalho de desenvolvimento.

Mas o que é o TDD?

O Desenvolvimento Orientado a Testes (TDD) é uma metodologia de desenvolvimento de software que consiste em escrever testes automatizados antes da implementação do código. O processo do TDD segue um ciclo de repetição composto por três etapas: "Red-Green-Refactor" (Vermelho-Verde-Refatoração).

A figura "Fluxo TDD" demonstra este processo:

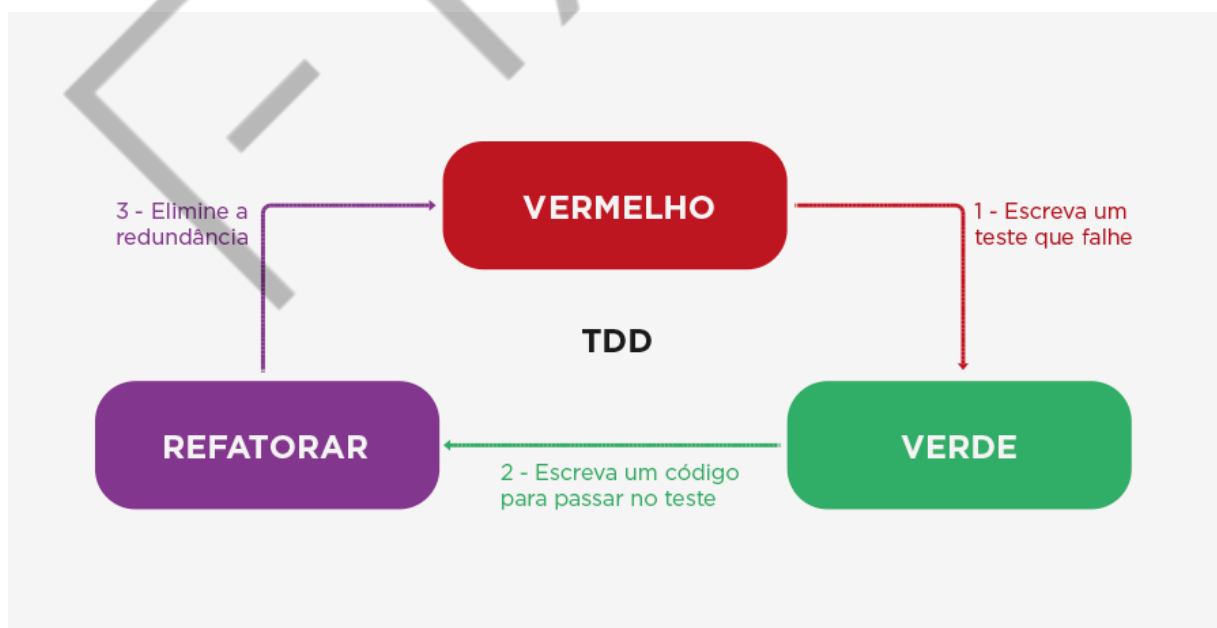


Figura 1 - Fluxo TDD
Fonte: Alves (2013)

Este ciclo Red-Green-Refactor é um pilar do Desenvolvimento Orientado a Testes (TDD), sendo crucial entender cada uma de suas etapas. Vamos explorá-las agora com exemplos práticos usando Jest para um projeto em JavaScript.

Red (Vermelho)

Na primeira fase, escrevemos um teste que esperamos que falhe, pois ele descreve um comportamento desejado do código que ainda não foi implementado. Vejamos como iniciar com um teste para uma função que verifica se uma string contém apenas caracteres alfabéticos:

```
const isAlphaString = require('./stringValidator');

test('isAlphaString should return true for an alphabetic string', () => {
  expect(isAlphaString("Hello")).toBeTruthy();
});
```

stringValidator.test.js

Neste estágio, o teste está "vermelho", pois a função `isAlphaString` ainda não existe, e esperamos que a string "Hello" seja considerada válida.

Green (Verde)

Na segunda fase, implementamos o mínimo de código necessário para fazer o teste passar:

```
function isAlphaString(input) {
  for (let c of input) {
    if (!c.match(/[a-zA-Z]/)) {
      return false;
    }
  }
  return true;
}

module.exports = isAlphaString;
```

Após adicionar esta implementação básica, o teste, quando executado, deve passar com sucesso. Isso indica que a funcionalidade de verificar se uma string é inteiramente alfabética foi atendida.

Refactor (Refatoração)

A etapa final é sobre melhorar o código sem alterar seu comportamento. Podemos refinar a nossa implementação usando expressões regulares de maneira mais eficiente:

```
function isAlphaString(input) {  
  return /^[a-zA-Z]+$/.test(input);  
}  
  
module.exports = isAlphaString;
```

stringValidator.js refactored

Depois da refatoração, rodamos os testes novamente para confirmar que a mudança não afetou o comportamento esperado, e o teste ainda passa.

Ao seguir o ciclo Red-Green-Refactor do TDD, garantimos que nosso código é testado de forma abrangente e que as funcionalidades são implementadas corretamente. A fase de refatoração nos permite melhorar o código sem afetar o comportamento esperado, resultando em um código mais legível, manutenível e confiável.

Até aqui tranquilo, não é mesmo?! Implementamos o fluxo em um código simples, mas, e se pensarmos em algo mais complexo, em que temos mais cenários de teste?

O fluxo seria o mesmo: independentemente dos cenários de teste, podemos utilizar o conceito do TDD com o fluxo Red -> Green -> Refactor, uma vez que esse fluxo, além de garantir que nosso código tem o mínimo de cobertura, permite que possamos testar e validar diferentes cenários de uso e refinar continuamente o código por meio da fase de refatoração.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu a importância dos testes de unidade, o ciclo Red-Green-Refactor, a abordagem de escrever testes antes da implementação e os benefícios do TDD, como maior confiabilidade do código, menor quantidade de defeitos e maior capacidade de refatoração.

Se você ficou com alguma dúvida ou quer aprender ainda mais sobre os conteúdos estudados, não deixe de acessar o Discord. Lá, você poderá conversar com docentes, colegas e pessoa de Community Management sempre que desejar. Explore!

REFERÊNCIAS

ALVES, L. G. **TDD** – Test-Driven Development. 2013. Disponível em: <micreiros.com/tdd-test-driven-development-2/>. Acesso em: 29 jun. 2023.

ANICHE, M. **Test-Driven Development**. Teste e Design no Mundo Real. [s.l.]: Casa do Código, 2012.

EXEMPLO

PALAVRAS-CHAVE

Palavras-chave: Testes. Qualidade de Software. TDD.

EXEMPLO

POS TECH